

Aplicación del Proceso Personal de Software en Cursos de Ingeniería de Software

Leonardo Bermón Angarita¹

lbermona@unal.edu.co

¹Departamento de Informática y Computación, Universidad Nacional de Colombia Sede Manizales, Campus La Nubia, Manizales, Caldas, Colombia

Pages: 501-516

Resumen: El Proceso Personal de Software es un proceso de desarrollo individual de software explícitamente diseñado para un aprendizaje gradual que brinde disciplina y mejora del proceso. El objetivo de este estudio es realizar un análisis de la aplicación de PSP en cursos universitarios de Ingeniería de Software evaluando su impacto en el desempeño en áreas como estimación, productividad, defectos y costo de la calidad. Durante siete periodos académicos, este curso se impartió en una asignatura universitaria donde los estudiantes recolectaron datos del proceso PSP mientras desarrollaban ejercicios de programación orientada a objetos. Los resultados muestran mejoras del proceso software tanto en estimación de tamaño como en el tiempo de desarrollo, sostenimiento de una productividad estable, reducción de la densidad de defectos y obtención de un costo de calidad aceptable mediante el uso de PSP.

Palabras-clave: Ingeniería de software; Proceso software; Medición de software; Proceso personal de software; Enseñanza informática.

Application of the Personal Software Process in Software Engineering Courses

Abstract: Personal Software Process is a software development process explicitly designed to provide a gradual learning oriented to discipline and process improvement. The aim of this study is to analyze the application of PSP in a university course of Software Engineering evaluating its impact on performance in areas such as estimating, productivity, defects and quality costs. During 8 semesters, this course given in a university and its students collected PSP process data while they developed object-oriented programming exercises. The results show improvements in both size and time estimating, sustaining a stable productivity, reduced defect density and obtaining of an acceptable quality cost using PSP.

Keywords: Software engineering; Software process; Software measurement; Personal Software Process, Computer education.

1. Introducción

El desafío de desarrollar productos software de alta calidad es substancial y de alta demanda hoy en día. Para resolver esta cuestión, el Instituto de Ingeniería de Software (*Software Engineering Institute* - SEI) desarrolló el Proceso Personal de Software (*Personal Software Process* - PSP) (Humphrey, 2005).

PSP es un proceso de desarrollo de software esencialmente interesado en desarrollar habilidades individuales en ingenieros de software para que aprendan a controlar y desarrollar su propio proceso y prácticas de trabajo (Soto & Rafael, 2020; Denwattana, Saengsai & Charoenchaimonkon, 2019).

PSP cubre cinco niveles del *Software Engineering Body of Knowledge* - SWBoK (Fairley, Bourque & Keppler, 2014): diseño, construcción, pruebas, proceso y calidad. Estos temas son cubiertos por PSP tanto en teoría como en la práctica.

En ambientes educativos, PSP puede ser aplicado como una herramienta bastante efectiva para dar a conocer conceptos acerca de modelos de procesos software orientados al plan, brindar disciplina a los estudiantes durante el desarrollo de software e identificar áreas de mejora de procesos (Joonbakhsh & Sami, 2018).

Este artículo analiza un conjunto de datos PSP recolectados durante varios periodos académicos por estudiantes de un programa universitario de informática. Los datos PSP cubrieron los niveles incluidos en PSP y fueron recolectados por medio de herramientas informáticas a medida que se desarrollaba un conjunto de ejercicios de programación orientada a objetos. El estudio realizado evalúa el impacto en el desempeño de los estudiantes en áreas como estimación de tiempo y tamaño, productividad, densidad de defectos y costo de la calidad.

El artículo está organizado de la siguiente manera: La segunda sección presenta una visión general sobre PSP describiendo sus beneficios, niveles constituyentes, elementos básicos y actividades del proceso. La tercera sección muestra un resumen de los principales trabajos encontrados en fuentes bibliográficas sobre la enseñanza de PSP. La cuarta sección describe la metodología del estudio realizado. La quinta sección presenta y discute los resultados obtenidos. Por último, la sexta sección proporciona las conclusiones y trabajo futuro.

2. Proceso Personal de Software

PSP fue desarrollado por Watss Humphrey en el SEI para ampliar el concepto de mejora del proceso software del nivel organizacional al nivel individual (Nasir et al., 2014).

PSP es un proceso definido, medido y diseñado para ser utilizado de manera individual por desarrolladores de software. PSP se enfoca en: (1) mejorar las prácticas y habilidades individuales de desarrollo de software; (2) identificar fácilmente áreas de mejora en el proceso y (3) permitir hacer estimaciones de tamaño, tiempo y defectos (Grütter & Ferber, 2002).

PSP sugiere actividades de medición y análisis consistentes para que los desarrolladores puedan identificar deficiencias del proceso y hacer una estimación confiable de esfuerzo y calidad (Pando & Rodríguez, 2018).

PSP está conformado por un conjunto de prácticas sólidas que generan un proceso disciplinado para desarrollar software. Las prácticas PSP se aplican en varios niveles de procesos donde cada nivel incluye un conjunto de actividades para el desarrollo de pequeños programas en forma individual (Fontana et al., 2015). Los niveles de PSP se muestran en la Figura 1. Cada nivel incorpora nuevas destrezas y técnicas para mejorar la calidad de software. La retroalimentación basada en la recolección de métricas ayuda al desarrollador a mejorar su propio proceso de desarrollo. Los elementos del proceso incluidos en cada nivel son incrementales; de tal manera que los elementos de un nivel superior incluyen los elementos de niveles inferiores. Los ítems dentro de cada nivel se refieren a aspectos del proceso aprendidos en dicho nivel. Se recomienda aplicar el proceso de manera secuencial iniciando con PSP0.

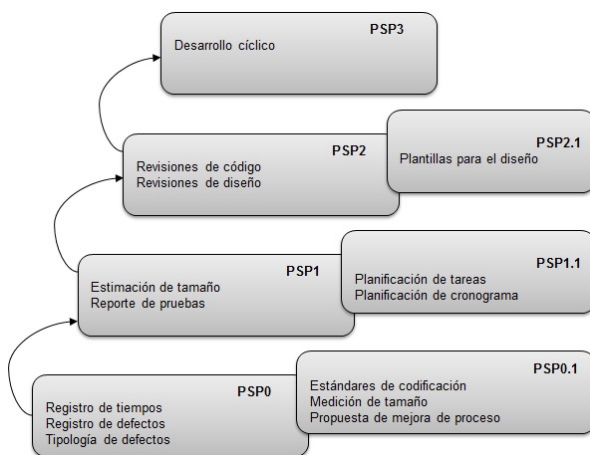


Figura 1 – Niveles de PSP. Fuente: (Humphrey, 2005),

Los niveles de PSP son los siguientes:

- PSP0: Línea base del proceso a partir de la recolección de datos de tiempos y defectos.
- PSP0.1: Incorpora estándares de codificación, estándares de conteo basados en líneas de código para la medición del tamaño de los programas y un mecanismo de propuesta para la mejora del proceso.
- PSP1: Incluye prácticas de estimación de tamaño y tiempo basada en el método PROBE (*Proxy Based Estimation* - Estimación basada en Proxy) utilizando datos históricos de proyectos. Además, incluye la realización de reportes de pruebas.
- PSP1.1: Involucra prácticas de planificación y seguimiento de proyectos utilizando la técnica del valor ganado.
- PSP2: Incluye aspectos de calidad del proceso software a partir de actividades como revisiones de diseño y de código basadas en el uso de listas de verificación. Además, se incorpora un conjunto de métricas para evaluar la calidad del proceso y del producto desarrollado.

- PSP2.1: Introduce plantillas para especificar en forma detallada el diseño de programas y técnicas para realizar la verificación del diseño.
- PSP3: Amplía el proceso para poder desarrollar programas grandes en forma individual aplicando un enfoque iterativo e incremental.

PSP contiene cuatro elementos básicos que están incluidos en los niveles del proceso personal y forman un marco de trabajo para organizar el trabajo del proyecto (Humphrey, 2005). Estos elementos se describen a continuación.

- Scripts: Son descripciones que guían la ejecución personal de un proceso especificando sus objetivos, criterios de entrada y salida, etapas a ejecutar, medidas y criterios de calidad.
- Formularios y logs: Proporcionan un medio para recolectar datos. Especifican los datos requeridos y dónde recolectarlos.
- Medidas: Cuantifican los procesos y productos permitiendo desarrollar perfiles de datos de proyectos previos que pueden ser utilizados para planificar, analizar procesos, determinar cómo mejorarlos y monitorear su ejecución.
- Estándares: Proporcionan definiciones consistentes y precisas que guían el trabajo y la recolección y uso de datos. Los estándares de codificación y conteo permiten realizar mediciones que son aplicadas uniformemente a través de múltiples proyectos.

El proceso PSP se detalla en la Figura 2 y está conformado por tres fases (Jirapanthong, 2017).

- Planificación: Se produce un plan de realización del trabajo.
- Desarrollo: Se realiza el trabajo. Esta fase se descompone en las actividades de: (a) diseño (se produce un diseño del programa); (b) revisión del diseño (se revisa el diseño y se corrigen defectos detectados); (c) codificación (se codifica el programa en un determinado lenguaje de programación); (d) revisión de

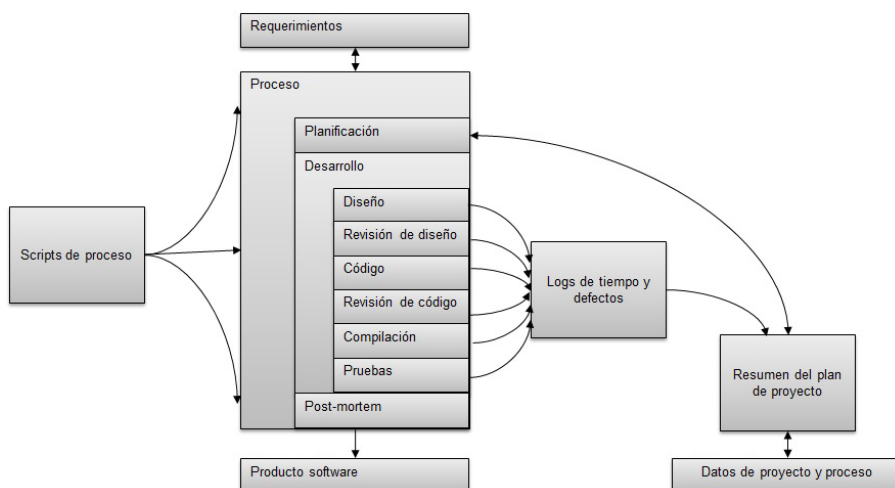


Figura 2 – Proceso PSP. Fuente: (Humphrey, 2005).

código (se revisa el código y se corrigen defectos detectados); (e) compilación (se compila el programa y se corrigen defectos detectados); y (f) pruebas (se prueba el programa y se corrigen defectos detectados).

- Post-mortem: Se compara el desempeño real con el desempeño planificado, se registran datos del proceso, se produce un resumen del plan de proyecto y se documentan ideas de mejora del proceso.

Las fases del proceso se ejecutan consultando los scripts. A medida que se ejecuta cada fase, se van recolectando datos a través de formularios y logs. Los datos del proyecto y del proceso se consolidan en un resumen de plan de proyecto. Este proceso se aplica a todos los niveles PSP. El proceso permitirá a partir de un conjunto de requisitos software, construir un producto software.

3. Trabajos Relacionados

Este trabajo está relacionado con diferentes propuestas educativas que incorporan el proceso PSP en currículos de Ingeniería de Software en diferentes universidades a nivel mundial.

La estrategia de búsqueda de estudios consistió en consultar en las siguientes fuentes bibliográficas: Science Direct, IEEE Xplore, Springer, Web of Science y ACM Digital Library en el área de Computer Science.

Las palabras claves utilizadas fueron:

(“Personal Software Process” or “PSP”) and (“education” or “learning” or “training” or “study” or “teaching”)

La búsqueda se realizó en artículos publicados entre los años 2000 y 2020, y se encontraron 94 publicaciones (eliminando duplicados) que cumplieron los criterios de búsqueda. Luego, después de leer los resúmenes correspondientes, la introducción y conclusiones de cada artículo, se seleccionó una cantidad total final de 20 artículos. Los artículos restantes no fueron seleccionados debido a que no estaban orientados al objetivo de este estudio, eran muy teóricos, no presentaban una buena calidad de datos o no eran extensos. En la Tabla 1 se presenta un listado de los trabajos encontrados, lugar e institución donde se aplicaron y características de los estudios.

Estudio	Características
<i>(Kamatar & Hayes, 2000)</i>	Servicios de Información Avanzada, Inc. (USA), Contexto: Curso de formación PSP de 2 semanas, 2 proyectos de desarrollo de software.
<i>(Lisack, 2000)</i>	Universidad de Purdue (USA), Contexto: 116 estudiantes de primer año durante dos semestres en cursos de introducción a la programación.
<i>(Wesslen, 2000)</i>	Universidad de Lund (Suecia), Contexto: 3 cursos, 131 personas; Evaluación: 10 ejercicios; Lenguajes: C/C++, Pascal, Simula, Java, Lisp, ADA.
<i>(Maletic, Howald & Marcus, 2001)</i>	Universidad de Memphis (USA), Contexto: Dos cursos de informática, estudiantes con experiencia diversa en programación.
<i>(Prechelt & Unger, 2001)</i>	Universidad de Karlsruhe (Alemania), Contexto: 2 grupos, PSP (29 personas) y no-PSP (19 personas) de 8º semestre; Evaluación: 10 ejercicios; Lenguaje: Java, C/C++, Módulo.

Estudio	Características
<i>(Runeson, 2001)</i>	Universidad de Lund (Suecia), Contexto: Estudiantes de primer año y estudiantes de posgrado; Evaluación: 10 ejercicios.
<i>(Abrahamsson & Kautz, 2002)</i>	Escuela de negocios de Copenhague (Dinamarca), Contexto: 22 estudiantes de cuarto y quinto año; Intensidad: 13 clases; Evaluación: 8 ejercicios; Lenguajes: Java, C++, Visual Basic.
<i>(Borstler et al., 2002)</i>	Varias universidades (Suecia y USA), Contexto: Estudiantes de 2/3 año. Se aplicaron dos versiones PSP (full y lite); Evaluación: series de ejercicios A y B de un libro de PSP.
<i>(Grutter & Ferber, 2002)</i>	Centro de investigación (Alemania), Contexto: 2 proyectos, Proyecto 1: (Gestión de workflow, 2 años, Java), Proyecto 2 (Sistema óptico 3D, C++, 5-10 personas).
<i>(Umphress & Hamilton, 2002)</i>	Universidad de Auburn (USA), Contexto: Curso de procesos software en 10 periodos académicos con 32 estudiantes.
<i>(Wohlin, 2004)</i>	Universidad de Lund (Suecia), Contexto: Curso con 65 personas; Evaluación: 10 ejercicios; Lenguaje: C.
<i>(Rombach et al., 2008)</i>	Instituto Fraunhofer (Alemania), Contexto: Datos de 3090 ingenieros; Evaluación: 10 ejercicios.
<i>(Lee, Baik & Peter, 2008)</i>	Universidad de Corea (Corea), Contexto: Muestra de 6 estudiantes; Intensidad: Curso de 24 horas; Evaluación: 8 ejercicios; Lenguaje: C.
<i>(Shena, Hsueha & Leeb, 2011)</i>	Taiwán, Contexto: Curso con 16 estudiantes; Intensidad: 18 semanas; Evaluación: 10 ejercicios; Lenguajes: Java, C++.
<i>(Rong et al., 2012)</i>	Instituto de Software de la Universidad de Nanjing (China), Contexto: 300 estudiantes finalizando el primer año; Intensidad: 10 días; Evaluación: 4 ejercicios; Lenguaje: Java.
<i>(Manrique & Anaya, 2012)</i>	Corporación Universitaria Adventista (Colombia), Contexto: 4 grupos de diferentes niveles; Evaluación: 2 prácticas por grupo.
<i>(Martinez et al., 2014)</i>	Universidad de Baja California (México), Contexto: Curso básico de programación, 140 estudiantes; Intensidad: 16 semanas, 3 horas de clase y 2 de laboratorio por semana; Evaluación: 12 ejercicios; Lenguaje: C.
<i>(Gasca-Hurtado et al., 2015)</i>	Empresa (Colombia), Contexto: 74 encuestados.
<i>(Ron, Zhang & Shao, 2016)</i>	Instituto de Software de la Universidad de Nanjing (China), Contexto: Curso adaptado de PSP (PSP+), 70 estudiantes finalizando el primer año; Evaluación: 8 ejercicios.
<i>(López-Martín, Nassif & Abran, 2017)</i>	Varias universidades (México, Canadá, China), Contexto: Curso adaptado de PSP, 181 estudiantes de una maestría; Intensidad: 12-16 semanas, Evaluación: 7 ejercicios, Lenguajes: C++ y Java.

Tabla 1 – Trabajos relacionados

Ninguno de los trabajos relacionados presenta datos PSP para periodos largos de tiempo como el presentado en este estudio. Generalmente los datos corresponden a un solo curso en un único periodo académico. (Kamatar & Hayes, 2000) y (Lisack, 2000) abarcan dos periodos consecutivos. Los más extensos fueron (Grutter & Ferber, 2002) y (Umphress & Hamilton, 2002) pero sin datos consolidados.

Con respecto a los sujetos involucrados en los estudios, presentan un espectro muy amplio con participación de estudiantes de primer año, de años intermedios o avanzados. Algunos estudios presentan datos con profesionales o estudiantes de posgrado. La aplicación de PSP a estudiantes de primer año es bueno para que comiencen a utilizar conceptos de ingeniería de software en etapas tempranas. Sin embargo, el interés fundamental debe ser aprender a programar. Por lo tanto, se pueden presentar desviaciones en el aprendizaje. En etapas avanzadas, se pueden focalizar en el proceso, pero se puede ofrecer resistencia debido a la alta exigencia y disciplina en recolectar datos del proceso.

El proceso PSP Full, definido por Humphrey, fue el más utilizado. (Lisack, 2000; Borstler et al., 2002; Manrique & Anaya, 2012; Rong, Zhang & Shao, 2016) trabajaron con versiones modificadas, simplificadas o extendidas del proceso original.

La gran mayoría de los trabajos incluye una recolección y análisis de métricas básicas de PSP. Los estudios analizan las estimaciones de tamaño y esfuerzo, el registro y eliminación de defectos y la productividad. Un tema no tan analizado es la estimación de defectos. También se presentan análisis cruzados con datos no tan comunes como el lenguaje programación (Wohlin, 2004; López-Martín, Nassif & Abran, 2017), la confiabilidad (Prechelt & Unger, 2001) o las notas obtenidas por los estudiantes (Rong et al., 2012). Generalmente los estudios concuerdan con los resultados obtenidos por Humphrey como: la distribución de defectos disminuye con cada nivel; se presentan sobre y sub-estimaciones de tamaño y tiempo, pero se reducen con el tiempo; y la productividad aumenta y se estabiliza rápidamente.

El estudio realizado en este artículo presenta un análisis de datos PSP con un horizonte amplio de su aplicación en varios periodos académicos, pero con características uniformes como los conceptos teóricos impartidos, sujetos con experiencia en programación de últimos años universitarios, una aplicación homogénea de PSP en todos los periodos y análisis de una métrica como el costo de calidad que no fue abordada por ningún estudio.

4. Metodología

El objetivo del estudio es evaluar el desempeño histórico de los estudiantes en estimación de tiempo y tamaño, productividad, densidad de defectos y costo de la calidad utilizando datos del proceso PSP.

4.1. Sujetos

Los datos fueron recolectados de un curso de PSP desarrollado en la Universidad Nacional de Colombia Sede Manizales incluido en una asignatura de Ingeniería de Software. Los datos corresponden a los ejercicios realizados por los estudiantes en siete periodos académicos comprendidos entre el primer semestre de 2011 hasta el primer semestre de 2018. En total participaron 91 estudiantes. Ninguno de ellos había realizado previamente un curso relacionado con PSP. La realización de los ejercicios PSP fue obligatoria.

Los estudiantes eran de cuarto y quinto año de la carrera de Administración de Sistemas Informáticos. Cada participante seleccionó el lenguaje de programación que quería usar

en las prácticas, según su experiencia. A los estudiantes se les aconsejó que el curso de PSP no era adecuado para aprender un nuevo lenguaje de programación. No hubo preferencia de lenguajes de programación, pero los más utilizados fueron Java, PHP y Python. Todos los estudiantes adoptaron un estándar de codificación y conteo de líneas de código.

4.2.Contexto de los cursos

El curso de PSP se desarrolló durante un periodo de 15 semanas con una intensidad semanal de 4 horas. (Humphrey, 2005) se utilizó como libro de referencia para el curso. Para el desarrollo de cada ejercicio de programación, los estudiantes tenían de una a dos semanas (dependiendo de su complejidad). Los estudiantes no fueron penalizados por su desempeño con respecto a la calidad de sus ejercicios (es decir, por obtener menores o mayores estimaciones o defectos). En su lugar, fueron penalizados si no seguían las pautas del proceso PSP.

Los participantes fueron orientados en el proceso PSP durante la realización de los ejercicios. Después de cada entrega, se revisaron sus datos y recibieron retroalimentación adecuada antes de iniciar el siguiente ejercicio. Los datos de PSP fueron recolectados por medio de la herramienta *PSP Student Workbook* o *PSP Dashboard* (Process Dashboard, 2021). El material educativo del curso y las prácticas desarrolladas fueron enviados a través de plataformas de e-learning como Blackboard (Blackboard, 2021) y Moodle (Moodle, 2021). El componente teórico del curso se presenta en la Tabla 2.

Semana-Tema	Contexto PSP	Semana-Tema	Contexto PSP
1-Introducción a procesos software	Introducción	2-Introducción a PSP	Introducción
3-Registro de tiempo y defectos	PSP0	4-Medición del tamaño	PSP0.1
5-Estándares de codificación y de conteo LOC	PSP0.1	6-Estimación de software	PSP1
7-Tabla de tamaños relativos	PSP1	8-Método PROBE	PSP1
9-Valor ganado	PSP1.1	10-Planificación y cronograma del proyecto	PSP1.1
11-Calidad de software	PSP2	12-Revisiones de diseño y de código	PSP2
13-Plantillas de diseño	PSP2.1	14-Verificación del diseño	PSP2.1
15-Desarrollo cíclico e iterativo	PSP3		

Tabla 2 – Componentes teóricos del curso de PSP

Al finalizar el curso, los estudiantes expusieron un reporte con un análisis de sus datos PSP. En la Tabla 3 se presenta la cantidad de estudiantes, prácticas y ejercicios desarrollados en cada periodo académico. No se realizaron ejercicios sobre PSP3 debido a que están orientados al desarrollo de grandes programas.

Periodo, estudiantes, prácticas	Ejercicios de programación por nivel PSP
Periodo: 1, Estudiantes: 19, Prácticas: 6	PSPo (operaciones con arrays), PSPo.1 (ordenamiento de arrays), PSP1 (array de inmuebles), PSP1.1 (ingreso de inmuebles), PSP2 (búsqueda de inmuebles), PSP2.1 (reportes de inmuebles)
Periodo: 2, Estudiantes: 18, Prácticas: 6	PSPo (registro de estudiantes), PSPo.1 (registro de asignaturas), PSP1 (registro de notas y promedios), PSP1.1 (reportes generales), PSP2 (reportes específicos), PSP2.1 (generación de archivos)
Periodo: 3, Estudiantes: 11, Prácticas: 6	PSPo (cálculo de media y desviación de una lista numérica), PSPo.1 (conteo de LOC), PSP1 (cálculos de regresión lineal), PSP1.1 (ordenamiento de datos de tamaño), PSP2 (cálculo de tabla de tamaños relativos), PSP2.1 (operaciones con listas)
Periodo: 4, Estudiantes: 23, Prácticas: 3	PSPo (sumas palíndromas), PSP1 (tabla de tamaños relativos), PSP2 (cálculo de notas de estudiantes)
Periodo: 5, Estudiantes: 5, Prácticas: 6	PSPo (cálculo de media y desviación de una lista numérica), PSPo.1 (cálculo de media y desviación a partir de archivos de texto), PSP1 (regresión lineal), PSP1.1 (tabla de tamaños relativos - distribución simple), PSP2 (tabla de tamaños relativos - distribución normal y log-normal), PSP2.1 (nómina de empleados)
Periodo: 6, Estudiantes: 7, Prácticas: 6	PSPo (números primos consecutivos), PSPo.1 (números primos gemelos), PSP1 (conjetura de Collatz), PSP1.1 (cadenas de caracteres), PSP2 (secuencias saltarinas), PSP2.1 (secuencias de sumandos)
Periodo: 7, Estudiantes: 8, Prácticas: 4	PSPo (criptografía), PSP1 (tercias), PSP1.1 (números divisibles por 11), PSP2.1 (departamento de ventas)

Tabla 3 – Ejercicios por periodo académico

Los ejercicios de programación fueron tomados de diversas fuentes: Ejercicios estándares del libro de PSP (Humphrey, 2005), libros de programación competitiva (Laaksonen, 2020), ejercicios básicos de programación de varias fuentes y de inspiración propia del instructor. En algunos periodos académicos se realizaron menos ejercicios debido a las habilidades y experiencia en programación de los estudiantes inscritos en el curso.

Con respecto a la evaluación de las prácticas desarrolladas, se definió un conjunto de criterios de evaluación donde se le dio bastante importancia a la correcta recolección de los datos del proceso de desarrollo individual. Se realizó un seguimiento semanal de los estudiantes, revisando sus datos PSP para detectar errores en los datos y poder corregirlos.

4.3. Recolección de datos

Los datos de las prácticas de cada estudiante, tomados de las herramientas *PSP Student Workbook* y *PSP Dashboard*, fueron consolidados por nivel PSP en una hoja de cálculo al finalizar cada periodo académico. El proceso PSP recolecta y calcula numerosas métricas para evaluar la calidad del producto y el esfuerzo de desarrollo. De los múltiples datos PSP, se seleccionaron los datos relacionados con la estimación de tiempo, estimación de

tamaño, productividad, densidad de defectos y costos de calidad debido a que son las métricas básicas del proceso PSP.

4.4. Análisis de datos

Los datos PSP recolectados fueron migrados a una herramienta de procesamiento estadístico. Por medio de esta herramienta, se generaron los respectivos diagramas de caja que representan gráficamente los datos numéricos de cada métrica por nivel a través de sus cuartiles. Los diagramas resultantes muestran la media, la mediana y los cuartiles de los datos, también permiten identificar valores atípicos.

5. Resultados y discusión

Los datos recolectados aplicando el proceso PSP están orientados tanto al producto como al proceso. En este estudio se presentarán los resultados enfocados en los siguientes aspectos: estimación de tiempo y tamaño, productividad, densidad de defectos y costos de calidad. Los datos corresponden a valores específicos de cada métrica recolectados y calculados a partir de los ejercicios realizados por los estudiantes en cada nivel PSP en todos los periodos académicos.

5.1. Estimación de tamaño

En PSP, las estimaciones de tamaño de los programas se realizan con la métrica de líneas de código (LOC), la cual es precisa, específica y puede automatizarse su conteo. La estimación de tamaño se realizó en todos los niveles de PSP, excepto en el nivel PSPO que no la tiene incluida. En la Figura 3 se presenta un diagrama de caja con los porcentajes de error de estimación de tamaño. Las estimaciones se estabilizan dentro de un 25% del margen de error y tendieron a aproximarse a la estimación real a medida que se realizaron más ejercicios ya que los primeros se realizaron sin datos históricos. Sin embargo, en la última práctica (PSP2.1) se observó un incremento en la subestimación de tamaño, el cual puede ser explicado debido al incremento del esfuerzo y de la cantidad de artefactos a desarrollar para finalizar este último nivel PSP, a pesar de que ya se cuentan con datos suficientes para aplicar un método PROBE tipo A o B de estimación de tamaño.

5.2. Estimación de tiempo

En la Figura 4 se presenta un diagrama de caja con los porcentajes de error de estimación de tiempo de los programas. Las estimaciones de tiempo se realizan de forma subjetiva (por tanteo o basado en experiencias previas) durante los niveles PSPO a PSP1.1. Para los dos últimos ejercicios ya se cuentan con por lo menos tres puntos de datos para aplicar en forma efectiva el método PROBE basado en datos históricos. Los valores medios positivos muestran sobre-estimaciones y los valores negativos indican sub-estimaciones. (Jørgensen, 2016) argumenta que las estimaciones de tiempo en horas de trabajo (en lugar de días de trabajo) tienden a ser subestimadas. Sin embargo, en este estudio, los datos de tiempo están en horas de trabajo y no se presenta dicha tendencia. En promedio, las sobre-estimaciones y sub-estimaciones fluctúan, pero con el tiempo se estabilizan y se acercan al valor real, mostrando que los estudiantes con la aplicación del método PROBE pueden hacer muy buenas estimaciones.

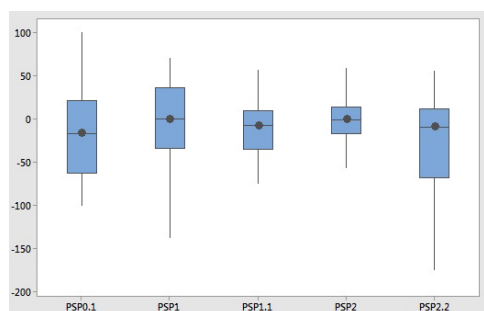


Figura 3 – Porcentajes de error de estimación de tamaño

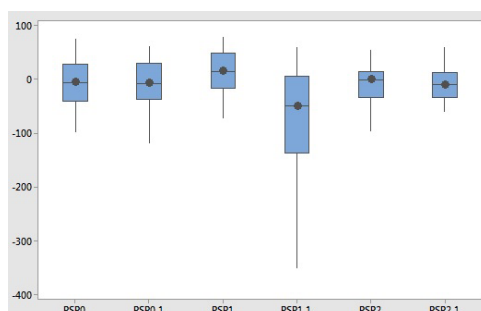


Figura 4 – Porcentajes de error de estimación de tiempo

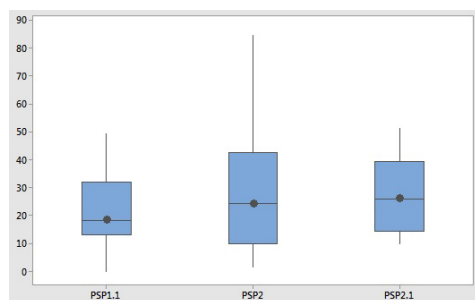


Figura 5 – Productividad en LOC/hora

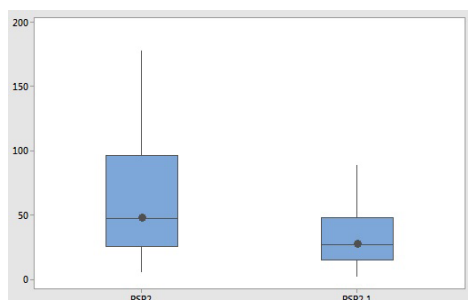


Figura 6 – Densidad de defectos medidas en defectos/KLOC

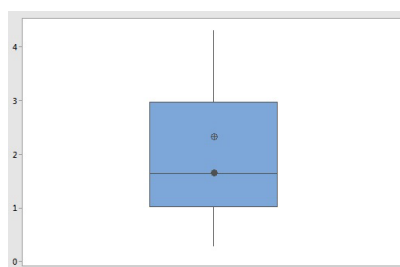


Figura 7 – Métrica AFR

Las estimaciones de tiempo fueron menos dispersas que las estimaciones de tamaño exceptuando el nivel PSP1.1. Una posible explicación a dicha dispersión puede ser el tiempo que tardaron los estudiantes en entender y aplicar los conceptos de valor ganado incluidos en PSP1.1 para realizar la planificación de tareas y cronograma. Las subestimaciones de tiempo son explicadas principalmente por las subestimaciones de tamaño de los programas realizadas previamente por los estudiantes ya que el tiempo dependerá del tamaño.

5.3. Productividad

La productividad de un estudiante se calcula como la cantidad de LOC desarrolladas por hora. Esta métrica se puede calcular a partir del nivel PSP0.1. Sin embargo, las herramientas informáticas la calculan a partir del nivel PSP1.1. En la Figura 5 se observa un diagrama de caja con la productividad obtenida por los estudiantes en las prácticas de PSP1.1, PSP2 y PSP2.1. La teoría sobre PSP resalta que la productividad no debe tender a aumentar o disminuir, si no a estabilizarse y mantenerse constante en el tiempo. Este comportamiento esperado es el que se observa en la figura, donde la productividad oscila entre 20 y 30 LOC/hora. Cabe resaltar que la productividad no fue afectada significativamente a medida que se avanza en cada nivel PSP, donde cada nivel requiere más artefactos a ser diligenciados por los estudiantes.

5.4. Densidad de defectos

La Figura 6 presenta la densidad de defectos calculada como la cantidad de defectos totales eliminados por miles de líneas de código (defectos/KLOC). Las herramientas informáticas calculan dichos valores automáticamente en los niveles PSP2 y PSP2.1. De acuerdo a los datos recolectados, se presenta una disminución de la densidad de defectos que se explica por la aplicación de las actividades de revisiones de diseño y de código que permiten que menos defectos se propaguen a etapas posteriores.

5.5. Costo de calidad

El costo de calidad en PSP se descompone en costo de fallas (*failure*) y costo de evaluación (*appraisal*). Una métrica de calidad que relaciona ambas es la tasa de evaluación/fallas (*appraisal to failure ratio* - AFR). La recomendación es que dicha tasa sea mayor o igual a 2.0 (Humphrey, 2005).

En la Figura 7 se presenta un diagrama de caja con el AFR obtenido por los estudiantes realizando el último nivel de PSP, PSP2.1, el cual incluye entre sus actividades el cálculo de dicha métrica. Se observa una mediana por debajo del valor recomendado, lo cual significa que los estudiantes están invirtiendo menos tiempo realizando revisiones de diseño y de código comparándolo con el tiempo de compilación y pruebas. Sin embargo, la media de valores sí cumple el AFR con un valor de 2,235. Estos resultados concuerdan con los presentados por (Rong et al., 2016) quienes manifiestan que los estudiantes pueden necesitar un apoyo adicional para lograr los objetivos y beneficios de PSP sobre la calidad del software.

5.6. Discusión

Algunos estudios recomiendan incluir PSP como elemento fundamental dentro de los planes curriculares de Ingeniería de Software y en introducir sus conceptos en los primeros cursos (Gasca-Hurtado et al., 2015). Sin embargo, PSP es un modelo de proceso software orientado al plan con muchos artefactos que permiten recolectar una gran cantidad de datos sobre el producto y el proceso. La recolección de dichos datos puede llevar a una sobrecarga cognitiva en estudiantes de primer año que están más interesados en aprender programación (cuestiones técnicas) que temas avanzados de

Ingeniería de Software (cuestiones del proceso). Por ello, se recomienda iniciar con una versión más ligera en cursos tempranos y una versión con todos los niveles PSP en cursos avanzados. En este estudio, todos los niveles de PSP se aplicaron con estudiantes de últimos años y con una gran diversidad de experiencia en programación.

Los estudiantes cometen muchos errores diligenciando los formularios PSP: colocan datos inconsistentes de tiempos y defectos y diligencian parcialmente los formularios. Para corregir estos defectos, se debe hacer un seguimiento inmediato una vez el instructor revise un ejercicio PSP. De tal manera que los estudiantes corrijan sus datos y los errores no se propaguen a niveles superiores PSP.

Con el caso de estudio desarrollado, se observó que el esfuerzo de PSP se refleja en programas con una mejor calidad y con menos defectos; el control y seguimiento de proyectos es mucho mejor; y los estudiantes conocen mejor sus fortalezas y debilidades en el proceso de desarrollo.

La experiencia de incluir PSP en el currículo ha sido gratificante, al permitir que los estudiantes refuercen sus técnicas de programación, conozcan y apliquen un proceso software definido y comiencen a plantear estrategias de mejora del proceso desde un punto de vista individual, para luego poder extrapolar dichos resultados a nivel grupal y organizacional. Además, los principales obstáculos que tuvo el curso son muy similares a los encontrados en la literatura: se requiere mucho tiempo para diligenciar los formularios PSP, en particular PSP2.1, que es el nivel superior.

De igual manera, los modelos de procesos software están continuamente reevaluándose. El auge de los procesos ágiles (Urbina et al., 2016) hace que PSP no se aplique en un 100% debido a la gran cantidad de artefactos utilizados. PSP debe ser adaptado dependiendo de los contextos organizacionales y convivir en entornos ágiles como Scrum o XP.

6. Conclusiones

Los datos recolectados en este estudio han validado los resultados esperados por la literatura relacionada, mostrando beneficios a futuros profesionales de software en áreas como: mejoras del proceso software tanto en estimación de tamaño como de tiempo, sostenimiento de una productividad estable, reducción de la densidad de defectos y obtención de un costo de calidad aceptable.

Los resultados son principalmente positivos en la educación sobre procesos software, aunque también se presentan críticas y recomendaciones al proceso. El curso básico propuesto por (Humphrey, 2005) ofrece 10 ejercicios estándares de programación. Sin embargo, cada institución puede cambiar su cantidad y requerimientos. Los resultados son más satisfactorios con estudiantes que tengan experiencia previa en programación. Los lenguajes de programación utilizados para el desarrollo de los ejercicios fueron orientados a objetos. Se recomienda la utilización de herramientas informáticas para diligenciar los formularios y logs, así como para realizar los cálculos de métricas del producto y del proceso.

El trabajo futuro estará orientado en lograr una mejora del proceso por medio de la combinación efectiva de procesos ágiles y orientados al plan (Shen, Rong & Shao, 2013)

y un análisis más exhaustivo de los datos recolectados aplicando técnicas de minería de datos y aprendizaje profundo.

Referencias

- Abrahamsson, P. & Kautz, K. (2002b). The personal software process: experiences from Denmark. *Proceedings 28th Euromicro Conference*, 367-374. Dormouth, Germany. <http://doi.org/10.1109/EURMIC.2002.1046223>
- Blackboard (2021). <http://www.blackboard.com>.
- Borstler, J., Carrington, D., Hislop, G.W. & Lisack, S. (2002). Teaching PSP: challenges and lessons learned. *IEEE Software*, 19(5), 42-48. <http://doi.org/10.1109/MS.2002.1032853>
- Denwattana, N., Saengsai, A., & Charoenchaimonkon, E. (2019). AppDOSI: An application for analyzing and monitoring the personal software process. *2019 4th International Conference on Information Technology (IncIT)*, 280-283. Bangkok, Thailand. IEEE. <http://doi.org/10.1109/INCIT.2019.8911993>
- Fairley, R. E. D., Bourque, P., & Keppler, J. (2014). The impact of SWEBOK Version 3 on software engineering education and training. *2014 IEEE 27th Conference on Software Engineering Education and Training (CSEET)*, 192-200. Klagenfurt, Austria. IEEE. <http://10.1109/CSEET.2014.6816804>
- Fontana, R. M., Meyer Jr, V., Reinehr, S., & Malucelli, A. (2015). Progressive Outcomes: A framework for maturing in agile software development. *Journal of Systems and Software*, 102, 88-108. <https://doi.org/10.1016/j.jss.2014.12.032>
- Gasca-Hurtado, G.P., Manrique-Losada, B., Gómez-Alvarez, M.C. & Arias, D.M. (2015). Diagnostic on teaching-learning of software design by using the Personal Software Process framework. *10th Iberian Conference on Information Systems and Technologies (CISTI)*, 1-7. Aveiro, Portugal. <http://doi.org/10.1109/CISTI.2015.7170385>
- Grutter, G. & Ferber, S. (2002). The Personal Software Process in practice: Experience in two cases over five years. *Software Quality - ECSQ 2002*, Lecture Notes in Computer Science, 2349, 165-174. Helsinki, Finland. http://doi.org/10.1007/3-540-47984-8_20
- Humphrey, W. S. (2005). *PSP(sm): A self-Improvement process for software engineers*. Pearson Education. Westford, Massachusetts, USA.
- Jirapanthong, W. (2017). Personal software process with automatic requirements traceability to support startups. *Journal of Reviews on Global Economics*, 6, 367-374. <http://doi.org/10.6000/1929-7092.2017.06.38>
- Joonbakhsh, A., & Sami, A. (2018). Mining and extraction of personal software process measures through IDE interaction logs. *Proceedings of the 15th International Conference on Mining Software Repositories*, 78-81. Gothenburg, Sweden. <https://doi.org/10.1145/3196398.3196462>

- Jørgensen, M. (2016). Unit effects in software project effort estimation: Work-hours gives lower effort estimates than workdays. *Journal of Systems and Software*, 117, 274-281. <http://doi.org/10.1016/j.jss.2016.03.048>
- Kamatar, J. & Hayes, W. (2000). An experience report on the personal software process. *IEEE Software*, 17(6), 85-89. <http://doi.org/10.1109/52.895173>
- Laaksonen, A. (2020). *Guide to competitive programming*. Springer International Publishing, Cham, Switzerland.
- Lee, T., Baik, D. & Peter, H. (2008). Cost Benefit Analysis of Personal Software Process Training Program. *IEEE 8th International Conference on Computer and Information Technology Workshops*, 631-636. Sydney, Australia. <http://doi.org/10.1109/CIT.2008.Workshops.120>
- Lisack, S.K. (2000). The Personal Software Process in the classroom: student reactions (an experience report). *Proceedings 13th Conference on Software Engineering Education & Training*, 169-175. Austin, USA. <http://doi.org/10.1109/CSEE.2000.827035>
- López-Martín, C., Nassif, A. B. & Abran, A. (2017). A training process for improving the quality of software projects developed by a practitioner. *Journal of Systems and Software*, 131, 98-111. <https://doi.org/10.1016/j.jss.2017.05.050>
- Maletic, J.I., Howald, A. & Marcus, A. (2001). Incorporating PSP into a traditional software engineering course: an experience report. *CSEET'01 Proceedings of the 14th Conference on Software Engineering Education and Training*, 89. Charlotte, NC, USA. <http://doi.org/10.1109/CSEE.2001.913825>
- Manrique, J.F.N. & Anaya, R. (2012). An empirical study of application of PSP methodology with students of a Systems Technology program with different levels of training. *XXXVIII Conferencia Latinoamericana en Informática (CLEI)*, 1-10. Medellín, Colombia. <http://doi.org/10.1109/CLEI.2012.6427134>
- Martinez, LG., Licea, G, Juarez, JR. & Aguilar, L. (2014). Experiences Using PSP and XP to Support Teaching in Undergraduate Programming Courses. *Computer applications in engineering education*, 22(3), 563-569. <http://doi.org/10.1002/cae.20581>
- Moodle (2021). <http://www.moodle.org>.
- Nasir, M. H. N. M., Alias, N. A., Fauzi, S. S. M., & Massatu, M. H. (2014). Implementation of the personal software process in academic settings and current support tools. *Software Design and Development: Concepts, Methodologies, Tools, and Applications*, 807-838. IGI-Global.
- Pando, B., & Rodríguez, G. (2018). PSP: how well it matches the skill set sought by the software industry in Latin America? A proposal to incorporate PSP into undergraduate SE education. *Proceedings of Jornadas Iberoamericanas de Ingeniería de Software e Ingeniería del Conocimiento (JIISIC 2018)*. Copiapo, Chile.

- Prechelt, L. & Unger, B. (2001). An experiment measuring the effects of personal software process (PSP) training. *IEEE Transactions on Software Engineering*, 27(5), 465-472. <http://doi.org/10.1109/32.922716>
- Process Dashboard. <http://www.processdash.com/>.
- Rombach, D., Müncha, J., Ocampo, A., Humphrey, W.S. & Burtonb, D. (2008). Software Process and Product Measurement. Teaching disciplined software development. *Journal of Systems and Software*, 81(5), 747-763. <http://doi.org/10.1016/j.jss.2007.06.004>
- Rong, G., Zhang, H., Xie, M. & Shao, D. (2012). Improving PSP education by pairing: An empirical study. *34th International Conference on Software Engineering (ICSE)*, 1245-1254. Zurich, Switzerland. <http://doi.org/10.1109/ICSE.2012.6227018>
- Rong, G., Zhang, H., Qi, S. & Shao, D. (2016). Can software engineering students program defect-free? An educational approach. *2016 IEEE/ACM 38th International Conference on Software Engineering Companion (ICSE-C)*, 364-373. Austin, Texas, USA. <http://doi.org/10.1145/2889160.2889189>
- Runeson, P. (2001). Experiences from teaching PSP for freshmen. *Proceedings 14th Conference on Software Engineering Education and Training*, 98-107. Charlotte, USA. <http://doi.org/10.1109/CSEE.2001.913826>
- Shen, M., Rong, G. & Shao, D. (2013). Integrating PSP with agile process: a systematic review. *Proceedings of the 2nd International Conference On Systems Engineering and Modeling (ICSEM-13)*, 0805-0811. Beijing, China. <http://doi.org/10.2991/icsem.2013.166>
- Shena, W., Hsueha, N. & Leeb, W. (2011). Assessing PSP effect in training disciplined software development: A Plan-Track-Review model. *Information and Software Technology*, 53(2), 137-148. <http://doi.org/10.1016/j.infsof.2010.09.004>
- Soto, B. C. P., & Rafael, G. R. (2020). Habilidades de Personal Software Process (PSP) para la industria del software en Latinoamérica. *Industrial data*, 23(1), 229-244. <https://doi.org/10.15381/idata.v23i1.17243>
- Umphress, D.A. & Hamilton, J.A. (2002). Software process as a foundation for teaching, learning, and accrediting. *Proceedings 15th Conference on Software Engineering Education and Training (CSEE&T 2002)*, 160-169. Covington, Kentucky, USA. <http://doi.org/10.1109/CSEE.2002.995208>
- Urbina, M. L., Abud, M. A., Peláez, G., Alor, G. y Sánchez, A. I. (2016). Propuesta de un modelo de integración de PSP y Scrum para mejorar la calidad del proceso de desarrollo en una MiPyME. *Research in Computing Science*, 120, 147-157.
- Wesslen. A. (2000). A Replicated Empirical Study of the Impact of the Methods in the PSP on Individual Engineers. *Empirical Software Engineering*, 5(2), 93-123. <http://doi.org/10.1023/A:1009811222725>
- Wohlin, C. (2004). Are Individual Differences in Software Development Performance Possible to Capture Using a Quantitative Survey?. *Empirical Software Engineering*, 9(3), 211-228. <http://doi.org/10.1023/B:EMSE.0000027780.08194.bo>

© 2021. This work is published under
<https://creativecommons.org/licenses/by-nc-nd/4.0/>(the
“License”). Notwithstanding the ProQuest Terms and
Conditions, you may use this content in accordance with the
terms of the License.